Техническое задание

Smart Video Encoder (`smart-video-encoder`)

Версия: 3.1
Статус: Draft / для утверждения
Дата: 2026-08-28
Целевая ОС P0: Ubuntu 24.04, Linux x86_64
Система сборки: CMake + vcpkg
CI/CD: GitLab CI
Основной сценарий: transcoding + adaptive recompression с приоритетом качества детектированных объектов.

---

1. Назначение

Разработать C++ библиотеку `smart-video-encoder` для аппаратного и программного кодирования видеопотока с адаптивным распределением качества по пространственным областям кадра.

Основная идея:

> Области кадра, содержащие объекты машинного зрения, должны кодироваться с более высоким приоритетом качества, чем фон, при сохранении заданной политики bitrate и latency.

Библиотека не выполняет детекцию объектов самостоятельно. Внешняя система передаёт для каждого кадра метаданные: bounding boxes, polygons, masks, track IDs, confidence и другие атрибуты. Библиотека преобразует эти данные в backend-specific ROI/QP/emphasis maps.

Проект должен поддерживать:

- offline transcoding;
- adaptive recompression;
- кодирование видеопотока камеры;
- CPU frames;
- `AVFrame`;
- CUDA-backed `AVFrame` / GPU memory;
- backend NVIDIA NVENC;
- backend Intel oneVPL;
- backend Rockchip MPP;
- программный fallback через FFmpeg software encoders.

---

2. Цели

2.1 Функциональные цели P0

1. Кодирование H.264/AVC.
2. Кодирование H.265/HEVC.
3. Object-first / ROI-aware compression.
4. Frame-level и region-level QP control там, где это поддерживается backend.
5. Поддержка нескольких ROI на кадр.
6. Temporal persistence ROI.
7. Сглаживание изменения качества между кадрами.
8. Передача пользовательской metadata, синхронизированной с PTS.
9. Работа с CPU и hardware frames.
10. Unified C++ API независимо от backend.
11. Capability discovery.
12. Fallback при отсутствии конкретной аппаратной функции.
13. Полная автоматическая сборка и тестирование в GitLab CI.
14. Воспроизводимый benchmark protocol.
15. Python wrapper с API, функционально эквивалентным основным сценариям публичного C++ API.

2.2 Архитектурные цели P1

1. Подготовка к H.266/VVC.
2. Возможность добавления AV1.
3. Дополнительные backend’ы без изменения публичного API.
4. Block/CTU-level quality map.
5. Multi-object priority scheduling.
6. Dynamic rate-budget allocation между ROI.

---

3. Не входит в scope

В P0 не входят:

- встроенный detector;
- встроенный OCR;
- tracker;
- обучение ML-моделей;
- собственный decoder;
- полноценный video container/muxing framework;
- Windows production support;
- собственная реализация H.264/H.265 codec;
- гарантированная идентичность битстрима между backend’ами.

Windows/Docker Desktop допускаются только как development/CI environment P2 и не являются production target.

---

4. Поддерживаемые кодеки

P0

Codec	Статус
H.264 / AVC	обязательно
H.265 / HEVC	обязательно

Архитектурно

Codec	Статус
H.266 / VVC	API-ready, backend optional
AV1	optional future backend
Другие	через расширяемый codec registry

Публичный API не должен содержать жёсткую зависимость от H.264/H.265-specific структур, если параметр может быть представлен общим интерфейсом.

---

5. Целевые backend’ы

5.1 NVIDIA

Основной путь:

```
AVFrame / CUDA frame
 | vsmart-video-encoder
 | vFFmpeg libavcodec
 | vh264_nvenc / hevc_nvenc
 | vNVENC
```

Требования:

- runtime capability detection;

- CUDA zero-copy, где технически возможно;
- использование backend-specific возможностей ROI/QP/emphasis map, если они доступны;
- отсутствие предположения, что одинаковые функции доступны на всех поколениях GPU.

5.2 Intel

Основной путь:

```
AVFrame
 | +--> FFmpeg QSV, если возможностей достаточно | +--> direct oneVPL adapt
```

Для ROI использовать capability query. Поддерживаемый максимум ROI и доступные режимы определяются runtime API.

При наличии `DeltaQP`:

- отрицательная коррекция QP повышает качество;
- координаты и карты должны быть приведены к ограничениям backend;
- библиотека не должна предполагать фиксированное число ROI.

5.3 Rockchip

Использовать Rockchip MPP напрямую через backend adapter.

Поддерживаемые P0:

- H.264;
- H.265;
- ROI;
- CTU/QP control, если доступно конкретному SoC/MPP;
- SEI/user data, если доступно.

Целевые SoC определяются capability layer. Минимальный hardware validation target должен включать один актуальный Rockchip SoC с H.264/H.265 encoder и ROI.

5.4 Software fallback

Через FFmpeg/libavcodec.

Назначение:

- CI;
- deterministic functional tests;
- отсутствие hardware;
- reference baseline.

Software backend не обязан обеспечивать все object-first возможности hardware backend. В capability report должен быть явно указан режим реализации.

---

6. Архитектура

```
+-----------------------------------------------------------+
|                     Application                           |
+-------------------------+---------------------------------+
 | v+-----------------------------------------------------------+
|                SmartVideoEncoder API                      |
|                                                           |
| EncoderConfig | Frame | ObjectMetadata | ROI Policy     |
+-------------------------+---------------------------------+
 | v+-----------------------------------------------------------+
|                 Adaptive Control Layer                    |
|                                                           |
| ROI Normalizer                                             |
| Priority Resolver                                          |
| Temporal ROI Manager                                       |
| Motion/Track Predictor                                     |
| QP / Quality Map Generator                                 |
| Bitrate Budget Controller                                  |
+-------------------------+---------------------------------+
 | v+-----------------------------------------------------------+
|                  Capability Layer                         |
| NVENC | oneVPL | Rockchip MPP | FFmpeg Software        |
+-------------------------+---------------------------------+
```

```
 | v+-----------------------------------------------------------+
|                  Backend Adapters                           |
|                                                             |
| NvencBackend                                                 |
| OneVplBackend                                                |
| RockchipMppBackend                                           |
| SoftwareBackend                                              |
+-------------------------------------------------------------+
 | v+-----------------------------------------------------------+
|                AVPacket / EncodedFrame                      |
+-------------------------------------------------------------+
```

---

7. Основные архитектурные принципы

7.1 Separation of policy and mechanism

Политика:

```
что считать важным
какой объект приоритетнее
насколько понизить QP
как долго сохранять ROI
какой bitrate budget выделить
```

не должна зависеть от конкретного encoder.

Механизм:

```
ROI rectangle
DeltaQP
QP map
CTU QP
emphasis map
backend fallback
```

реализуется backend adapter.

7.2 Capability-driven design

Каждый backend возвращает:

```
struct EncoderCapabilities {
 bool available; bool h264; bool h265; bool vvc;
 bool roi_rectangles; bool delta_qp; bool qp_map; bool ctu_qp; bool emphasi
 bool sei_user_data; bool cuda_frames; bool zero_copy_input;
 uint32_t max_roi_regions; uint32_t qp_map_block_width; uint32_t qp_map_blo
 bool cbr; bool vbr; bool low_latency;};
```

Запрещается silently fallback без записи в telemetry.

8. Публичный API

8.1 Создание encoder

```
namespace sve {

class SmartVideoEncoder {
public:
 static std::unique_ptr<SmartVideoEncoder> Create(const EncoderConfig& conf
 virtual ~SmartVideoEncoder() = default;
 virtual Status SubmitFrame( const AVFrame* frame, const FrameMetadata& met
 virtual Status ReceivePacket( EncodedPacket* packet) = 0;
 virtual Status Flush() = 0;
 virtual EncoderCapabilities GetCapabilities() const = 0;
 virtual EncoderStatistics GetStatistics() const = 0;};

}
```

8.2 Конфигурация

```
enum class Codec {
 H264, H265, VVC};

enum class Backend {
 Auto, NvidiaNvenc, IntelOneVpl, RockchipMpp, FfmpegSoftware};

enum class RateControl {
 CBR, VBR, CQ, CQP, BackendDefault};

struct EncoderConfig {
 Codec codec = Codec::H265; Backend backend = Backend::Auto;
 uint32_t width = 0; uint32_t height = 0;
 AVRational frame_rate{30, 1};
 uint64_t target_bitrate = 0; uint64_t max_bitrate = 0; uint64_t vbv_buffer
 uint32_t gop_size = 60; uint32_t max_b_frames = 0;
 RateControl rate_control = RateControl::VBR;
 bool low_latency = true;
 AdaptiveConfig adaptive; TemporalROIConfig temporal_roi; MetadataConfig me
```

---

9. Object metadata API

```
enum class ObjectPriority : uint8_t {
 Background = 0, Low, Normal, High, Critical};

struct BoundingBox {
 float x; float y; float width; float height;};

struct Polygon {
 std::vector<PointF> points;};

struct ObjectMetadata {
 uint64_t object_id = 0; uint64_t track_id = 0;
 BoundingBox bbox; std::optional<Polygon> polygon;
 float confidence = 1.0f;
 ObjectPriority priority = ObjectPriority::Normal;
 std::string class_name;
 std::optional<float> desired_qp_delta;};
```

```
struct FrameMetadata {
 int64_t pts = AV_NOPTS_VALUE; std::vector<ObjectMetadata> objects;};
```

Координаты могут быть:

- pixel coordinates;
- normalized coordinates.

Единый формат задаётся `MetadataCoordinateSystem`.

---

10. ROI pipeline

Для каждого кадра:

```
Input objects
 | vValidation
 | vCoordinate normalization
 | vClip to frame
 | vMinimum ROI expansion
 | vPriority resolution
 | vOverlap merge / split
 | vTemporal prediction
 | vSpatial smoothing
 | vTemporal smoothing
 | vBackend quantization
 | vROI rectangles / QP map / CTU map
```

---

11. ROI policy

11.1 Object-first policy

P0 policy:

```
Critical > High > Normal > Low > Background
```

Рекомендуемые logical QP offsets:

Priority	Default logical delta
Critical	-8
High	-6
Normal	-4
Low	-2
Background	+0 … +6

Это **logical policy**, а не прямое требование к hardware QP.

Backend adapter обязан:

1. ограничить диапазон;
2. преобразовать знак/единицы, если требуется;
3. записать applied values в telemetry.

11.2 Minimum ROI size

```
64 x 64 pixels
```

или больше, если backend требует alignment.

Малые объекты расширяются симметрично, но:

- не выходят за границы кадра;
- не перекрывают unrelated ROI без merge policy.

11.3 ROI margin

По умолчанию:

```
8 pixels
```

до backend alignment.

Должны поддерживаться:

- fixed margin;
- relative margin;
- class-specific margin.

---

12. Overlapping objects

При пересечении ROI применяется:

```
max importance wins
```

Для QP:

```
minimum QP wins
```

При необходимости допускается weighted composition:

```
priority score
+
confidence
+
object size
+
track age
```

Алгоритм выбора должен быть deterministic.

---

13. Temporal ROI

```
struct TemporalROIConfig {
 bool enabled = true;
 uint32_t max_missing_frames = 3;
 float confidence_decay = 0.85f;
 float spatial_smoothing = 0.25f; float temporal_smoothing = 0.35f;
 bool use_track_prediction = true;
 float min_roi_tracking_iou = 0.50f;
 uint32_t max_roi_lag_frames = 3; uint32_t max_roi_lag_ms = 100;
 int max_qp_delta_per_frame = 2;};
```

13.1 Поведение

Если объект временно пропал:

```
detected
detected
MISS
MISS
detected
```

ROI сохраняется до `max_missing_frames`.

Если объект не возвращается:

- priority плавно уменьшается;
- QP correction плавно возвращается к background;
- запрещены резкие quality jumps сверх `max_qp_delta_per_frame`.

---

14. ROI lag

Для GT object определяется:

```
t_gt = момент появления или существенного изменения объекта
```

Первый момент:

```
t_roi
```

при котором:

```
IoU(applied ROI, target ROI) >= 0.50
```

Тогда:

```
lag_frames = frame(t_roi) - frame(t_gt)
lag_ms = PTS(t_roi) - PTS(t_gt)
```

Acceptance:

- P95 <= 3 frames и <= 100 ms;
- P99 <= 6 frames и <= 200 ms.

---

15. Bitrate policy

Основной режим:

```
Object-first VBR
```

Цель:

> При ограниченном bitrate сначала сохранить task quality объектов, затем качество фона.

15.1 Primary benchmark profile

```
pixel_format: yuv420p
bit_depth: 8

gop_size: 60
keyint_min: 60
scene_cut: disabled
```

```yaml
b_frames: 0

rate_control: VBR

lookahead: disabled
spatial_aq: disabled
temporal_aq: disabled
```

В baseline и adaptive должны отличаться только механизмы adaptive spatial allocation.

15.2 Secondary random-access profile

```yaml
gop_size: 60
b_frames: 3
lookahead: 16
```

Значения должны быть идентичны baseline/adaptive.

---

16. Bitrate acceptance

Для matched-bitrate comparison:

```
|adaptive_avg_bitrate - baseline_avg_bitrate|
---------------------------------------------- <= 2%
baseline_avg_bitrate
```

Дополнительно:

```
average bitrate overshoot <= 5%
```

Rolling window:

```
window = 1 second
```

```
P95 rolling bitrate <= 120% target
```

---

17. Входные форматы

P0:

```
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
AV_PIX_FMT_P010
```

Дополнительно допускается:

- RGB/BGR input через conversion path;
- CUDA hardware frames;
- другие hardware frames через FFmpeg hwcontext.

При hardware input библиотека должна предпочитать zero-copy или hardware mapping. Необязательное скачивание GPU frame в RAM запрещено, если backend способен принять совместимый hardware frame.

---

18. AVFrame ownership

Библиотека:

- не изменяет ownership caller frame;
- выполняет reference, если кадр должен жить асинхронно;
- освобождает все внутренние references;
- не сохраняет raw pointer после завершения соответствующей операции без `av_frame_ref`.

---

19. Metadata

19.1 Внутренний metadata channel

Каждый encoded packet может сопровождаться:

```
struct EncodedFrameMetadata {
 int64_t input_pts; int64_t output_pts;
 std::vector<ObjectMetadata> objects;
 AppliedROIInfo roi_info;
 uint64_t frame_index;};
```

19.2 Bitstream metadata

P0 должна поддерживаться возможность backend-specific insertion:

- H.264 SEI user data unregistered;
- H.265 SEI user data unregistered;
- external sidecar metadata.

Формат payload:

```
version
frame_id
pts
object_count
objects[]
crc
```

Рекомендуемый P0 serialization:

- CBOR или FlatBuffers;
- versioned schema;
- bounded payload size.

19.3 Синхронизация

Ключ синхронизации:

1. PTS;
2. frame index;
3. optional UUID.

Acceptance:

```
100% metadata-to-frame synchronization
```

на functional metadata corpus.

---

20. FFmpeg как нижний слой

FFmpeg является основным abstraction layer для:

- `AVFrame`;
- `AVPacket`;
- pixel formats;
- hardware device contexts;
- hardware frame contexts;
- software fallback;
- NVENC/QSV integration;
- parsing/bitstream utilities.

Прямой backend API допускается там, где FFmpeg:

- не предоставляет требуемый ROI/QP mechanism;
- скрывает capability;
- не даёт требуемую CTU/QP control;
- не обеспечивает нужный metadata insertion;
- не позволяет zero-copy path.

---

21. Backend interface

```
class IEncoderBackend {
public:
 virtual ~IEncoderBackend() = default;
 virtual Status Open(const EncoderConfig&) = 0;
 virtual EncoderCapabilities QueryCapabilities() = 0;
 virtual Status Encode( const AVFrame* frame, const AppliedAdaptiveFrameCon
 virtual Status Flush() = 0;
 virtual BackendStatistics Statistics() const = 0;};
```

`AppliedAdaptiveFrameConfig` является внутренним canonical representation:

```
struct AppliedAdaptiveFrameConfig {
 std::vector<ResolvedROI> rois;
 std::optional<QPMap> qp_map;
 bool force_intra = false;
 FrameQualityBudget budget;};
```

22. Backend fallback matrix

Requested feature	Backend supports	Поведение
ROI rectangles	yes	native ROI
ROI rectangles	no, QP map yes	synthesize map
QP map	no, ROI yes	quantize to ROI
Neither	no	frame-level fallback или `Unsupported`
CTU control	yes	native CTU map
Metadata SEI	no	sidecar/external metadata

Fallback должен быть:

- deterministic;
- logged;
- доступен через statistics;
- тестируем.

---

23. Capability report

CLI/API:

```
smart-video-encoder --capabilities
```

Пример:

```
{
  "backend": "nvidia-nvenc",
  "device": "...",
  "codecs": ["h264", "h265"],
  "roi_rectangles": true,
  "qp_map": false,
  "emphasis_map": true,
  "max_roi_regions": 8
}
```

---

24. Threading model

P0:

```
Application thread
 | vInput queue
 | vAdaptive worker
 | vEncoder backend
 | vOutput queue
```

Требования:

- bounded queues;
- configurable queue depth;
- backpressure;
- flush correctness;
- EOS correctness;
- thread-safe statistics.

---

25. Error model

Использовать:

```
enum class StatusCode {
 Ok, Again, InvalidArgument, Unsupported, DeviceUnavailable, BackendError,
```

Ошибки backend должны содержать:

- backend;
- codec;
- native error code;
- human-readable message.

---

26. CMake и vcpkg

26.1 Требования

Сборка должна использовать:

- CMake;

- vcpkg manifest mode;
- C++20.

Структура:

```
smart-video-encoder/
├── CMakeLists.txt
├── CMakePresets.json
├── vcpkg.json
├── vcpkg-configuration.json
├── cmake/
├── include/
│   └── sve/
├── src/
│   ├── core/
│   ├── roi/
│   ├── temporal/
│   ├── backends/
│   │   ├── nvenc/
│   │   ├── onevpl/
│   │   ├── rockchip/
│   │   └── ffmpeg/
│   └── metadata/
├── tests/
├── benchmarks/
├── python/
│   ├── src/
│   ├── tests/
│   └── smart_video_encoder/
├── tools/
├── docker/
├── docs/
└── .gitlab-ci.yml
```

26.2 Options

```
option(SVE_ENABLE_NVENC "Enable NVIDIA backend" ON)
option(SVE_ENABLE_ONEVPL "Enable Intel oneVPL backend" ON)
option(SVE_ENABLE_ROCKCHIP "Enable Rockchip MPP backend" ON)
option(SVE_ENABLE_SOFTWARE "Enable software backend" ON)

option(SVE_BUILD_TESTS "Build tests" ON)
```

```
option(SVE_BUILD_BENCHMARKS "Build benchmarks" ON)
option(SVE_BUILD_TOOLS "Build CLI tools" ON)
option(SVE_BUILD_PYTHON "Build Python bindings" OFF)
```

Backend dependency должна быть optional, чтобы CI x86 без hardware мог собрать core и software backend.

---

27. Docker

27.1 Базовые образы

Минимум:

```
docker/Dockerfile.dev
docker/Dockerfile.runtime
docker/Dockerfile.test
docker/Dockerfile.nvidia
docker/Dockerfile.intel
docker/Dockerfile.rockchip
```

27.2 NVIDIA runtime

Контейнер использует NVIDIA Container Toolkit/runtime.

Требования:

- GPU должен быть доступен контейнеру;
- driver остаётся на host;
- userspace libraries должны быть совместимы с host driver/runtime.

Проверки:

- `nvidia-smi`;
- FFmpeg encoder discovery;
- encode/decode smoke test.

27.3 Intel

Для Linux hardware acceleration контейнер должен получать соответствующие DRM/render devices, обычно через `/dev/dri`, в зависимости от используемого stack и deployment.

Проверки:

- device enumeration;
- oneVPL implementation discovery;
- H.264/H.265 encode smoke test.

27.4 Rockchip

На target board контейнер должен получать необходимые device nodes и userspace MPP libraries согласно конкретной BSP/SoC configuration.

P0 не должен хардкодить один device path в portable API. Конкретный deployment profile документируется отдельно.

---

28. Observability

Каждый frame может генерировать:

```
frame_index
pts

object_count
roi_count

logical_qp_delta
applied_qp_delta

backend_mode
fallback_mode
```

```
input_timestamp
adaptive_timestamp
encode_start
encode_end

packet_size
instantaneous_bitrate
```

Агрегированные метрики:

- FPS;
- average latency;
- P50/P95/P99 encode latency;
- bitrate;
- average ROI count;
- fallback count;
- dropped frames;
- queue depth.

---

29. Логирование

Использовать structured logging.

Уровни:

- ERROR;
- WARN;
- INFO;
- DEBUG;
- TRACE.

Обязательные события:

- backend selected;
- capability report;
- fallback;

- unsupported feature;
- encoder reset/reconfigure;
- frame drop;
- metadata failure.

---

30. Тестовые уровни

```
L0  Static analysis / formatting
L1  Unit
L2  Integration
L3  Functional codec
L4  Hardware smoke
L5  Synthetic adaptive quality
L6  Natural video quality
L7  CV/task-aware
L8  Performance/stability
L9  Acceptance
```

---

31. Unit tests

Покрыть:

ROI

- clipping;
- normalization;
- alignment;
- minimum size;
- overlap;
- priority resolution;

- deterministic ordering.

Temporal

- detector miss;
- object reappearance;
- timeout;
- prediction;
- smoothing;
- ROI lag calculation.

Bitrate

- bpppf;
- rolling bitrate;
- overshoot;
- matching.

Metadata

- serialization;
- deserialization;
- schema version;
- CRC;
- PTS mapping.

Минимум:

- 100% unit tests pass;
- новые production modules не принимаются без unit tests для основной логики.

---

32. Functional codec tests

Для каждого доступного backend:

Test	H.264	H.265
Open/close	required	required
Encode 1 frame	required	required
Encode 300 frames	required	required
Flush	required	required
EOS	required	required
Monotonic PTS	required	required
Frame count	required	required
Decode output	required	required
Metadata	required	required

Acceptance:

- frame count decoded = frame count input;
- PTS mapping корректен;
- битстрим декодируется reference decoder;
- memory leaks отсутствуют в software/CPU CI job.

---

33. Synthetic benchmark corpus

Synthetic corpus обязателен и version-controlled.

Структура:

```
benchmarks/synthetic/
├── flat/
├── gradients/
├── checkerboards/
├── sine/
├── zoneplates/
├── slanted_edges/
├── moving_roi/
├── appearing_roi/
├── disappearing_roi/
├── crossing_objects/
├── small_objects/
├── temporal_flicker/
└── mixed_frequency/
```

33.1 Spatial frequency patterns

Обязательны:

- uniform fields;
- horizontal/vertical gradients;
- checkerboard;
- sinusoidal gratings;
- multi-frequency sine patterns;
- zone plate;
- slanted edge;
- text/grid pattern.

Цель:

- проверить распределение ошибок по spatial frequency;
- обнаружить block artifacts;
- проверить edge preservation;
- проверить leakage качества за пределы ROI.

33.2 Synthetic ROI patterns

Обязательные сценарии:

1. статический ROI;
2. ROI moving horizontally;
3. ROI moving diagonally;
4. ускоряющийся ROI;
5. появление;
6. исчезновение;
7. crossing ROI;
8. partial overlap;
9. маленький объект;
10. много ROI.

---

34. Synthetic ROI quality metrics

Primary:

```
ROI-VMAF
```

Secondary:

- ROI-PSNR-Y;
- ROI-SSIM;
- ROI-MS-SSIM;
- Background-VMAF;
- Global-VMAF.

ROI mask перед измерением расширяется на:

```
8 pixels
```

с последующим clipping.

Для baseline operating points:

```
ROI-VMAF in [60, 95]
```

Acceptance

```
mean Delta ROI-VMAF >= +2.0
median Delta ROI-VMAF >= +1.0
```

Не более 10% sequence × bitrate-point комбинаций могут иметь:

```
ROI-VMAF loss > 1.0
```

---

35. ROI effectiveness test

Synthetic sequence:

```
low-detail background
+
high-frequency ROI
```

При одинаковом target bitrate:

Если доступен actual QP:

```
mean QP_ROI <= mean QP_background - 2
```

Если actual QP недоступен:

```
ROI-VMAF - Background-VMAF >= +3
```

на специально подготовленном ROI pattern.

---

36. Natural video datasets

36.1 MOT17

Используется для:

- multi-object tracking;
- HOTA;
- IDF1;
- DetA;
- AssA;
- MOTA;
- ID switches.

Pipeline:

1. исходное видео;
2. baseline encode/decode;
3. adaptive encode/decode;
4. одинаковый detector/tracker;
5. MOT evaluation;
6. paired comparison.

Acceptance

```
Delta HOTA >= -0.5 pp
Delta IDF1 >= -0.5 pp
```

При matched bitrate.

36.2 Critical tracks

Subset:

```
track length >= 30 frames
median bbox area <= 5% frame area
```

Primary:

- Critical-IDF1;

- Critical-DetRe.

Acceptance:

```
Delta Critical-IDF1 >= -0.25 pp
Delta Critical-DetRe >= -0.25 pp
```

На R1/R2 хотя бы одна метрика:

```
Delta Critical-IDF1 >= +0.5 pp
```

или:

```
Delta Critical-DetRe >= +0.5 pp
```

36.3 UFPR-ALPR

Используется для:

- plate detection;
- OCR;
- small/high-frequency ROI;
- ALPR task quality.

Versioned benchmark bundle:

```
benchmarks/alpr/
├── detector/
│   ├── manifest.yaml
│   └── model.onnx
├── ocr/
│   ├── manifest.yaml
│   └── model.onnx
└── pipeline/config.yaml
```

Каждая модель:

```yaml
name:
version:
git_commit:
```

```
weights_sha256:
container_digest:
```

Primary:

- Exact Plate Match Rate.

Secondary:

- Character Accuracy;
- CER;
- Plate Detection Recall.

Acceptance:

```
Delta ExactPlateMatch >= -0.25 pp
Delta PlateDetectionRecall >= -0.25 pp
```

На R1/R2 хотя бы одна:

```
mean Delta ExactPlateMatch >= +0.5 pp
```

или:

```
mean Delta PlateDetectionRecall >= +0.5 pp
```

---

37. Дополнительные datasets

P1 рекомендуется добавить:

- BDD100K — detection;
- Cityscapes / KITTI — object-centric urban scenes;
- UVG / Xiph / standard video sequences — natural video RD quality;
- собственный surveillance corpus — при наличии прав на использование.

Лицензии, версии и split должны фиксироваться в dataset manifest.

---

38. Baseline calibration

Перед финальным acceptance выполняется calibration run.

Для каждого:

```
backend
codec
hardware class
resolution
dataset
R1..R4
```

создаётся baseline manifest.

Пример:

```
protocol: SVENC-1.0

backend: nvenc
codec: h265

encoder:
 version: exact-runtime-version preset: p4 rate_control: vbr target_bitrate
input:
 width: 1920 height: 1080 fps: 30
```

Изменение любого поля создаёт новый profile version.

39. Operating points

Resolution	R1	R2	R3	R4
720p30	0.75 Mbps	1.5 Mbps	3 Mbps	6 Mbps
1080p30	1.5 Mbps	3 Mbps	6 Mbps	12 Mbps

Resolution	R1	R2	R3	R4
1080p60	3 Mbps	6 Mbps	12 Mbps	24 Mbps
4K30	6 Mbps	12 Mbps	24 Mbps	48 Mbps

Дополнительно рассчитывать:

```
bpppf = bitrate / (width * height * fps)
```

---

40. RD and Task-RD analysis

Обязательные curves:

```
Bitrate vs Global-VMAF
Bitrate vs ROI-VMAF
Bitrate vs HOTA
Bitrate vs IDF1
Bitrate vs Critical-IDF1
Bitrate vs ExactPlateMatch
```

Для четырёх operating points.

Дополнительно:

- BD-Rate;
- BD-quality.

Нельзя принимать решение только по одному bitrate point.

---

41. Statistical protocol

Для каждого результата:

```
mean
median
stddev
P5
P95
95% confidence interval
```

Для baseline/adaptive paired comparison:

- paired bootstrap CI.

Каждый report обязан содержать:

```
benchmark_id
protocol_version
git_commit
build_id

backend
encoder_version
driver_version
firmware_version
hardware_id

dataset_version

detector_model_hash
ocr_model_hash

metrics_tool_version
```

---

42. Performance tests

Измерять:

```
encode FPS
end-to-end FPS

P50/P95/P99 encode latency
```

```
P50/P95/P99 end-to-end latency

CPU usage
GPU utilization where available

GPU memory
RAM

queue depth
packet rate
```

Acceptance

Для каждого certified hardware profile задаётся отдельный performance manifest.

Нельзя использовать единый FPS threshold для NVIDIA, Intel и Rockchip.

Пример:

```yaml
profile: nvenc-a5000-h265-1080p30
minimum_realtime_factor: 1.0
target_realtime_factor: 2.0
```

---

43. Stability tests

43.1 Soak

Минимум:

- 1 час continuous encode в CI smoke profile;
- 8 часов nightly hardware profile;
- 24 часа release-candidate soak на reference hardware.

Проверки:

- memory growth;
- frame loss;
- PTS;
- deadlock;
- backend reset;
- metadata synchronization.

43.2 Acceptance

```
unexpected crash = 0
deadlock = 0
frame accounting mismatch = 0
metadata mismatch = 0
unbounded memory growth = 0
```

---

44. GitLab CI

Pipeline:

```
lint
 |build
 |unit
 |integration
 |software-codec
 |synthetic
 |dataset-quality
 |hardware-nvidia
 |hardware-intel
 |hardware-rockchip
 |benchmark-report
 |acceptance
```

44.1 Jobs

lint

- clang-format;
- clang-tidy;
- cmake format.

build

Matrix:

- Debug;
- Release;
- GCC;
- Clang.

unit

- CTest;
- JUnit XML.

integration

- software backend;
- mocked capability layer.

synthetic

- generate corpus;
- encode baseline/adaptive;
- calculate metrics.

hardware jobs

Tagged runners:

```
nvidia
intel
```

```
rockchip
```

Hardware job должен:

1. собрать capability report;
2. сохранить environment manifest;
3. выполнить smoke;
4. выполнить selected quality suite.

---

45. GitLab artifacts

Сохранять:

```
artifacts/
├── binaries/
├── junit/
├── coverage/
├── logs/
├── environment/
├── capability/
├── encoded/
├── decoded/
├── metrics/
├── plots/
├── reports/
└── manifests/
```

Reports:

- JUnit;
- coverage;
- benchmark JSON;
- Markdown summary;
- CSV metrics.

---

46. Benchmark reproducibility

Каждый acceptance run должен быть воспроизводим через:

```
smart-video-benchmark run \ --protocol SVENC-1.0 \ --profile profiles/nvenc

Результат:

```text
results/<benchmark_id>/
```

Внутри:

```
environment.json
profile.yaml
dataset_manifest.json
model_manifest.json
metrics.json
summary.md
```

47. CI runners

x86 generic

Используется для:

- core;
- software;
- unit;
- integration;
- synthetic lightweight tests.

NVIDIA
```

Используется для:

- NVENC;
- CUDA frame path;
- H.264/H.265 smoke;
- selected benchmark.

Intel

Используется для:

- oneVPL;
- hardware encode;
- ROI capability tests.

Rockchip

Self-hosted ARM runner:

- native build;
- MPP;
- H.264/H.265;
- ROI/CTU tests;
- smoke benchmark.

---

48. Функциональные acceptance tests

AT-001 Build

```
cmake configure = PASS
build = PASS
unit tests = PASS
```

AT-002 H.264

Каждый certified backend:

- encode;
- decode;
- frame count;
- PTS;
- EOS.

AT-003 H.265

То же.

AT-004 ROI normalization

Набор edge cases:

- negative;
- outside frame;
- zero area;
- tiny object;
- overlap.

Ожидаемый результат deterministic.

AT-005 Temporal miss

ROI сохраняется:

```
missing <= max_missing_frames
```

Преждевременное исчезновение = FAIL.

AT-006 Metadata

100% frame mapping.

AT-007 Capability fallback

Requested unsupported feature:

- explicit `Unsupported` либо documented fallback;
- silent degradation = FAIL.

---

49. Quality acceptance matrix

Category	Metric	Requirement
Bitrate	Average difference	<= 2%
Bitrate	Average overshoot	<= 5%
Bitrate	P95 1-second rolling	<= 120% target
Synthetic ROI	Mean ΔROI-VMAF	>= +2.0
Synthetic ROI	Median ΔROI-VMAF	>= +1.0
Synthetic ROI	ROI regressions > 1 VMAF	<= 10%
ROI effect	QP difference	>= 2 when observable
MOT17	ΔHOTA	>= -0.5 pp
MOT17	ΔIDF1	>= -0.5 pp
Critical tracks	ΔCritical-IDF1	>= -0.25 pp
Critical tracks	ΔCritical-DetRe	>= -0.25 pp

Category	Metric	Requirement
MOT R1/R2	positive critical gain	>= +0.5 pp at least one
Temporal ROI	P95 lag	<= 3 frames and <= 100 ms
Temporal ROI	P99 lag	<= 6 frames and <= 200 ms
ALPR	ΔExactPlateMatch	>= -0.25 pp
ALPR	ΔPlateDetectionRecall	>= -0.25 pp
ALPR R1/R2	positive gain	>= +0.5 pp at least one
Functional	Frame count	exact
Functional	Metadata mapping	100%
Functional	Unit/regression tests	100% pass
Stability	Crash/deadlock	0

`pp` = percentage points.

---

50. PASS/FAIL rules

Acceptance считается успешным только если:

1. Все L0-L4 обязательные тесты PASS.
2. Все обязательные hardware tests для заявленного backend PASS.
3. Synthetic ROI criteria PASS.
4. Temporal ROI criteria PASS.
5. Task-aware criteria для применимого dataset PASS.
6. Bitrate constraints PASS.
7. Нет silent fallback.
8. Все manifests и versions присутствуют.
9. Benchmark protocol version совпадает с утверждённой.
10. Dataset/model hashes соответствуют manifest.

Если backend не поддерживает функцию:

```
SKIP
```

разрешён только при capability evidence.

```
catch(error) -> continue -> PASS
```

запрещено.

---

51. Deliverables

Разработчик обязан предоставить:

1. Исходный код.
2. Public headers.
3. CMake project.
4. vcpkg manifest.
5. Dockerfiles.
6. GitLab CI.
7. Unit/integration/functional tests.
8. Synthetic generator.
9. Benchmark runner.
10. Dataset manifests.
11. Model manifests.
12. Quality report generator.
13. Capability CLI.
14. API documentation.
15. Backend support matrix.
16. Example applications.
17. Performance reports.
18. Acceptance report.
19. Python package, type stubs, examples, tests и wheel artifacts.

---

52. Example applications

```
examples/
├── encode_cpu.cpp
├── encode_cuda.cpp
├── adaptive_boxes.cpp
├── adaptive_tracks.cpp
├── metadata_sei.cpp
└── transcode.cpp
```

Минимальный пример:

```
sve::EncoderConfig config;
config.codec = sve::Codec::H265;
config.backend = sve::Backend::Auto;

config.target_bitrate = 6'000'000;
config.gop_size = 60;

config.adaptive.enabled = true;

auto encoder = sve::SmartVideoEncoder::Create(config);

for (;;) {
 AVFrame* frame = GetFrame();
 sve::FrameMetadata metadata; metadata.pts = frame->pts; metadata.objects =
 encoder->SubmitFrame(frame, metadata);
 sve::EncodedPacket packet; while (encoder->ReceivePacket(&packet).ok()) {
```

53. Рекомендуемая структура benchmark repository

```
benchmarks/
├── protocol/
│   ├── SVENC-1.0.md
```

```
│   └── svenc-1.0.yaml
├── profiles/
│   ├── common/
│   ├── nvenc/
│   ├── onevpl/
│   └── rockchip/
├── synthetic/
├── datasets/
│   ├── mot17.yaml
│   ├── ufpr-alpr.yaml
│   └── natural-video.yaml
├── models/
│   ├── detector.yaml
│   └── ocr.yaml
├── scripts/
├── results/
└── reports/
```

---

54. Этапы реализации

Milestone 1 — Core

- public API;
- AVFrame;
- software backend;
- ROI normalization;
- metadata core;
- Python wrapper и packaging;
- CMake/vcpkg;
- GitLab CI.

Milestone 2 — NVIDIA

- H.264;
- H.265;

- CUDA input;
- capability detection;
- adaptive ROI path;
- hardware tests.

Milestone 3 — Intel

- oneVPL adapter;
- H.264/H.265;
- ROI DeltaQP;
- capability query;
- Docker/runtime validation.

Milestone 4 — Rockchip

- MPP adapter;
- H.264/H.265;
- ROI;
- CTU QP where available;
- ARM GitLab runner.

Milestone 5 — Benchmark

- synthetic corpus;
- VMAF/PSNR/SSIM/MS-SSIM;
- MOT17;
- UFPR-ALPR;
- Task-RD reports.

Milestone 6 — Release candidate

- soak;
- regression;

- performance;
- acceptance report.

---

55. Definition of Done

Версия 1.0 считается завершённой, если:

- библиотека собирается на Ubuntu 24.04;
- используется CMake + vcpkg;
- GitLab CI полностью автоматизирован;
- реализованы H.264 и H.265;
- реализованы NVIDIA, Intel oneVPL и Rockchip MPP backend'ы согласно capability matrix;
- software fallback доступен;
- AVFrame является обязательным входным интерфейсом;
- CPU YUV/RGB path работает;
- CUDA path работает на certified NVIDIA profile;
- object metadata преобразуется в ROI/QP policy;
- temporal ROI работает;
- metadata синхронизирована с frame/PTS;
- Python wrapper предоставляет основные encode, ROI/metadata, capability и statistics сценарии;
- Python wheels собираются и проходят тесты в поддерживаемой Python matrix;
- synthetic benchmark проходит acceptance;
- MOT17 и UFPR-ALPR проходят применимые task criteria;
- baseline profiles versioned;
- benchmark results reproducible;
- все обязательные GitLab CI stages PASS;
- release candidate проходит stability suite.

---

56. Нормативные технические источники для реализации

Проект должен опираться прежде всего на официальную документацию конкретного backend и фиксировать точную версию API/SDK в build manifest.

Основные источники:

- FFmpeg API и codec documentation;
- NVIDIA Video Codec SDK / NVENC documentation;
- Intel oneVPL API;
- Rockchip MPP documentation и исходный код;
- Netflix VMAF;
- MOTChallenge/MOT evaluation tooling;
- документация GitLab CI/CD.

При изменении версии backend API необходимо повторно выполнить capability discovery и regression benchmark.

---

57. Итоговое архитектурное решение

Рекомендуемая модель:

```
+-----------------+Detector / Tracker ->| Object Metadata|
+--------+-------+ | v +----------------+ | Smart Policy   | | ROI / Prior
```

Ключевое требование:

> Публичная логика object-first compression должна быть backend-independent, а различия NVENC/oneVPL/Rockchip должны быть изолированы в capability-driven adapters.

Это позволяет:

- использовать один API для камер и transcoding;
- не привязывать бизнес-логику к одному GPU vendor;
- постепенно добавлять VVC/AV1;
- сравнивать backend'ы единым benchmark protocol;
- доказывать пользу adaptive compression не только через PSNR/VMAF, но и через downstream CV metrics.

---

58. Python wrapper

58.1 Обязательность и назначение

Для библиотеки должен быть реализован официальный Python package `smart-video-encoder`, импортируемый как `smart_video_encoder`. Wrapper входит в P0 и предназначен для интеграции с detector/tracker pipelines, benchmark tooling, Jupyter и Python-сервисами без дублирования encoder logic на Python.

Python layer должен быть тонким: кодирование, ROI policy, temporal state, capability mapping, metadata synchronization и backend fallback остаются в C++ core. Python wrapper отвечает за преобразование типов, lifetime, исключения, GIL и удобный Pythonic API.

58.2 Выбор технологии

Рассмотрены два основных варианта:

Критерий	pybind11	Cython
Интеграция с современным C++20 API	прямая, natural binding классов, enum, STL и smart pointers	требует `.pyx` / `.pxd`, C/C++ declarations и дополнительного generated layer

Критерий	pybind11	Cython
CMake integration	native CMake target	отдельный Cython generation/build step
Ownership и RAII	хорошо соответствует `unique_ptr`, `shared_ptr` и C++ destructors	возможно, но требует более явного управления wrapper objects
NumPy/buffer protocol	встроенная поддержка массивов и buffers	зрелая typed-memoryview/NumPy integration
Исключения	прямой mapping и custom exception translators	ручное описание и преобразование ошибок
GIL	явный scoped release/call guards	зрелые `nogil` sections
Поддерживаемость небольшого C++ API	меньше boilerplate, bindings находятся рядом с C++ типами	выгоднее при большом Python-first numeric layer или тесной обвязке C API

Выбор P0: pybind11.

Причины:

1. Core является C++20 библиотекой с классами, enum, RAII и backend-independent API, а не C API.
2. Проект уже использует CMake; pybind11 естественно добавляется отдельным CMake target.
3. Binding layer должен оставаться тонким и не содержать алгоритмической логики.
4. pybind11 упрощает binding `EncoderConfig`, metadata structures, capabilities, statistics и encoder lifetime.
5. NumPy/buffer protocol и явное освобождение GIL достаточны для CPU frame path и длительных encode operations.

Cython не выбирается как основной binding generator. Он допускается в будущем только для изолированного Python-first preprocessing module, если profiling покажет измеримую пользу typed memoryviews или generated C loops. Публичный Python API не должен зависеть от выбранного внутреннего binding technology.

При выборе учитываются подходы похожих native media/computer-vision проектов: PyAV применяет Cython для глубокой обвязки FFmpeg C API, тогда как для небольшого class-oriented C++20 API целесообразнее прямой binding через pybind11. Нормативными источниками реализации должны быть официальные руководства pybind11, Cython, CPython buffer protocol, NumPy и Python packaging.

58.3 Python API P0

Wrapper должен предоставлять:

```
import smart_video_encoder as sve

config = sve.EncoderConfig(
 codec=sve.Codec.H265, backend=sve.Backend.AUTO, width=1920, height=1080, t

with sve.SmartVideoEncoder(config) as encoder:
 encoder.submit_frame(frame, metadata) packets = encoder.receive_available(
```

Обязательные bindings:

- `SmartVideoEncoder` с context manager и deterministic `close()`;
- `EncoderConfig`, `AdaptiveConfig`, `TemporalROIConfig`, `MetadataConfig`;
- `Codec`, `Backend`, `RateControl`, `ObjectPriority`, `StatusCode`;
- `BoundingBox`, `Polygon`, `ObjectMetadata`, `FrameMetadata`;
- `EncodedPacket`, `EncodedFrameMetadata`;
- `EncoderCapabilities`, `EncoderStatistics`;
- `submit_frame`, `receive_packet` / `receive_available`, `flush`, `get_capabilities`, `get_statistics`;
- module-level capability discovery без обязательного открытия encoder;
- Python exception hierarchy, сохраняющая `StatusCode`, backend, codec, native error code и исходное сообщение.

58.4 Frame interoperability

P0 должен поддерживать CPU frames через NumPy и Python buffer protocol:

- planar YUV420P;
- NV12;
- P010 с корректным dtype/stride;
- явное описание width, height, pixel format, PTS, plane offsets и strides;
- non-contiguous arrays только при валидных strides; иначе `ValueError`;
- read-only input, если C++ encoder не должен изменять frame memory.

Для каждого input wrapper обязан валидировать dtype, shape, strides, alignment, plane sizes и lifetime до передачи в C++.

Zero-copy разрешён только если lifetime Python owner гарантирован до завершения асинхронной C++ операции. Binding должен удерживать strong reference на owner либо создавать безопасный `AVFrame` reference с корректным release callback. Если гарантия невозможна, выполняется явная копия, отражаемая в statistics/telemetry. Скрытое использование dangling pointer запрещено.

Опциональная P1 interoperability:

- PyAV `VideoFrame` через документированный adapter, без зависимости C++ core от private PyAV ABI;
- CUDA Array Interface и/или DLPack для совместимых CUDA buffers;
- CuPy/PyTorch adapters поверх стабильного buffer/DLPack contract.

Wrapper не должен принимать произвольный integer как raw `AVFrame*` или device pointer в safe public API. Unsafe interop допускается только отдельным явно обозначенным expert API.

58.5 GIL, threading и callbacks

- GIL должен освобождаться на потенциально блокирующих вызовах `submit_frame`, `receive_packet`, `flush`, ожидании очередей и backend encode operations.
- GIL нельзя удерживать на протяжении hardware encode или ожидания output queue.
- Перед созданием Python objects, преобразованием exception или вызовом Python callback GIL должен быть захвачен повторно.

- C++ worker threads не должны вызывать Python callbacks после уничтожения wrapper/interpreter shutdown.
- P0 не должен зависеть от callbacks из encode hot path; предпочтительны polling/iterator/queue APIs.
- Thread-safety Python API должна соответствовать гарантиям C++ API и быть явно документирована.

58.6 Ownership и error mapping

- Python object владеет C++ encoder через RAII-compatible holder.
- `close()` и context manager должны быть idempotent.
- После `close()` операции возвращают предсказуемую Python exception.
- `EncodedPacket` должен владеть или reference-count безопасно удерживать packet data; view не может пережить owner.
- C++ exceptions не должны пересекать CPython ABI boundary.
- `StatusCode::Again` и `EndOfStream` должны иметь документированную Python semantics; они не должны маскироваться как неизвестный `RuntimeError`.
- Ошибки invalid shape/dtype/strides отображаются в `ValueError` / `TypeError`; backend failures — в специализированные `SmartVideoEncoderError` subclasses.

58.7 Packaging

Использовать:

- `pyproject.toml`;
- `scikit-build-core` как PEP 517 build backend;
- pybind11 CMake target;
- package layout `python/smart_video_encoder`;
- type hints и `.pyi` stubs либо эквивалентную typed API документацию;
- wheel build для поддерживаемых CPython versions на Linux x86_64 и Linux aarch64;
- source distribution, если она воспроизводимо собирается с документированными system dependencies.

Wheel не должен незаконно или технически небезопасно vendor'ить GPU drivers, NVIDIA kernel components либо host-specific hardware runtime. Политика bundling FFmpeg/oneVPL/MPP shared libraries должна быть документирована с учётом ABI, лицензий и wheel repair. При использовании системных libraries import-time diagnostics должны сообщать отсутствующую library и ожидаемую ABI/version.

Python и C++ package versions должны происходить из одного version source. Build manifest Python package должен включать git commit, build ID, compiler, C++ ABI, FFmpeg/backend versions и enabled features.

58.8 CMake и структура

Рекомендуемая структура:

```
python/
├── pyproject.toml
├── CMakeLists.txt
├── src/
│   ├── module.cpp
│   ├── encoder_bindings.cpp
│   ├── config_bindings.cpp
│   ├── metadata_bindings.cpp
│   ├── numpy_frame.cpp
│   └── exception_translation.cpp
├── smart_video_encoder/
│   ├── __init__.py
│   └── py.typed
├── tests/
└── examples/
```

Python extension собирается только при `SVE_BUILD_PYTHON=ON`. C++ core не должен зависеть от Python, NumPy или pybind11. Binding target линкуется с публичной C++ library и не обращается к private backend internals.

58.9 Тестирование и CI

Обязательные Python tests:

- import и version/build manifest;

- создание/закрытие encoder и context manager;
- config/enums/metadata round-trip;
- NumPy YUV420P/NV12/P010 shape, dtype и stride validation;
- ownership/lifetime tests, включая удаление исходного NumPy owner после submit;
- encode/receive/flush/EOS на software backend;
- H.264/H.265 decode validation и точный frame count;
- PTS и metadata mapping 100%;
- exception mapping для всех `StatusCode`;
- GIL concurrency test: другой Python thread продолжает выполняться во время blocking encode/flush;
- repeated create/destroy и leak checks;
- wheel installation test в чистом virtual environment;
- hardware smoke для NVENC/oneVPL/Rockchip wheels на tagged runners.

GitLab CI matrix должна включать все поддерживаемые CPython minor versions, Debug/Release extension build, Linux x86_64 и Linux aarch64. Минимально поддерживаемые версии Python фиксируются в release manifest; на момент релиза выбираются поддерживаемые upstream версии CPython, а не бессрочно захардкоженный диапазон.

58.10 Acceptance Python wrapper

Python wrapper принимается, если:

1. Основные C++ и Python сценарии дают функционально эквивалентные packets, PTS, metadata и capability reports при одинаковой конфигурации.
2. Software encode/decode H.264/H.265 проходит для всей Python CI matrix.
3. Некорректные buffer shape/dtype/strides отклоняются детерминированно без crash или memory corruption.
4. Ownership tests не обнаруживают use-after-free, double-free или утечки references.
5. Blocking native calls освобождают GIL.
6. Установка wheel в clean environment и `import smart_video_encoder` проходят без build tree.
7. Hardware-specific отсутствие runtime library или device возвращает диагностируемую Python exception, а не import crash.

8. Wrapper overhead измеряется отдельно от encode latency; для CPU frame submit без обязательной копии P95 overhead не должен превышать 5% относительно эквивалентного C++ вызова на том же профиле.
9. API reference, type hints и минимум три примера поставляются вместе с package: CPU NumPy encode, adaptive boxes и capability discovery.

Приложение A. Рекомендуемые профили

A.1 Object-first primary

```
profile: object-first-primary-v1

codec: h265
rate_control: vbr

gop_size: 60
b_frames: 0

lookahead: false
spatial_aq: false
temporal_aq: false

adaptive:
 enabled: true policy: object-first
```

A.2 Baseline

```
profile: baseline-primary-v1

codec: h265
rate_control: vbr

gop_size: 60
b_frames: 0

lookahead: false
spatial_aq: false
temporal_aq: false
```

```
adaptive:
 enabled: false
```

Приложение B. Обязательные manifest поля

```
protocol_version:
benchmark_id:

library:
 git_commit: build_id:
backend:
 name: version:
hardware:
 model: driver: firmware:
encoder:
 codec: preset: rate_control: target_bitrate: max_bitrate: vbv: gop: b_fram
input:
 width: height: fps: pixel_format:
dataset:
 name: version: manifest_sha256:
models:
 detector: name: version: sha256: ocr: name: version: sha256:
metrics:
 vmaf_version: evaluator_version:
```